IMPLEMENTATION GUIDE

Intégration CRYPTONEO

Signature électronique des baux et des mandats de gestion

Mon Toit CI · Plateforme de location immobilière — Côte d'Ivoire
Branche 4.0.0 · Plateforme CRYPTONEO / ANSUT (signature.ansut.ci)
Version 1.0 — 2 novembre 2026

Table des matières

1. Résumé exécutif 1

2. Vue d'ensemble 2

2.1 Contexte et objectifs 2

2.2 La plateforme CRYPTONEO / ANSUT 2

2.3 Périmètre de l'intégration 3

3. Architecture technique 3

3.1 Principe directeur : secret côté serveur uniquement 3

3.2 Le contrat partagé (initiative DAT-004) 3

3.3 Gestion du jeton JWT 4

4. Configuration et prérequis 4

4.1 Variables d'environnement 4

4.2 Modèle de données 5

4.3 Checklist de mise en configuration 5

5. Cycle de vie de la signature électronique 6

5.1 Étape 1 — Génération du certificat numérique 6

5.2 Étape 2 — Demande et envoi du code OTP 7

5.3 Étape 3 — Signature du document (signFileBatch) 7

5.4 Étape 4 et 5 — Vérification et archivage 8

5.5 Déclinaison baux : signature à deux avec activation du bail 8

5.6 Déclinaison mandats : double signature propriétaire / agence 8

6. Cartographie des composants du projet 9

6.1 Edge Functions Supabase 9

6.2 Routes API Next.js 9

6.3 Bibliothèques, services et hooks 9

6.4 Composants d'interface utilisateur 10

7. Endpoints CRYPTONEO consommés.....10

7.1 Format de la requête de signature (SignRequestItem).....10

8. Sécurité.....11

8.1 Authentification des appels.....11

8.2 Cycle de vie de l'OTP.....11

8.3 Codes de retour CRYPTONEO.....11

8.4 Journalisation d'audit et notifications.....12

9. Gouvernance du code et tests de parité.....12

9.1 Règle d'or DAT-004.....12

9.2 Divergences historiques corrigées.....12

9.3 Tests de parité à rejouer.....13

10. Exploitation, dépannage et recommandations.....13

10.1 Erreurs courantes et résolutions.....13

10.2 Rappels automatiques des signatures en attente.....13

10.3 Recommandations d'évolution.....14

10.4 Documents de référence internes.....14

Remarque : cette table des matières est générée par champs. Après toute modification du document, faites un clic droit sur la table puis « Mettre à jour les champs » pour actualiser la pagination.

1. Résumé exécutif

Le présent document décrit l'implémentation complète de la signature électronique CRYPTONEO au sein de la plateforme Mon Toit CI (branche 4.0.0). Cette intégration permet aux utilisateurs — propriétaires, locataires et agences — de signer à distance, avec une valeur juridique renforcée, les deux types d'actes centraux de la plateforme : les baux de location (lease_contracts) et les mandats de gestion immobilière (mandats, agency_mandates). Elle s'appuie sur la plateforme ANSUT opérée par CRYPTONEO (signature.ansut.ci), accessible au moyen d'une API REST authentifiée par clés d'application.

L'architecture retenue repose sur un principe non négociable : aucune clé d'application (appKey, appSecret) n'est exposée côté client. Toutes les communications avec CRYPTONEO transitent par deux couches serveur — les routes API Next.js et les Edge Functions Supabase (runtime Deno) — qui partagent un contrat commun, le module cryptoneo-contract.ts, source de vérité unique des codes de succès, des types et des règles de normalisation (initiative DAT-004 de déduplication).

Le cycle de vie de signature s'articule en cinq étapes : génération d'un certificat numérique par utilisateur, envoi d'un code OTP à usage unique (SMS ou e-mail), signature par lots du document (signFileBatch) associée à un alias de certificat, vérification asynchrone du statut de l'opération (operationId), puis archivage du document signé dans Supabase Storage. Chaque étape est journalisée (admin_audit_logs), notifiée aux parties concernées et contrôlée par des politiques de sécurité Row Level Security (RLS) au niveau des données.

Ce document constitue la référence technique de l'intégration : architecture, configuration, flux détaillés, cartographie exhaustive des composants, règles de sécurité, gouvernance du code partagé et guide de dépannage. Il s'adresse aux développeurs, DevOps et auditeurs de la plateforme.

2. Vue d'ensemble

2.1 Contexte et objectifs

Mon Toit CI digitalise la relation locative en Côte d'Ivoire, de la publication des biens jusqu'à la gestion des quittances. Deux documents de nature contractuelle exigeaient une signature dématérialisée opposable : le bail de location, conclu entre le propriétaire et le locataire, et le mandat de gestion, conclu entre le propriétaire et l'agence immobilière. Avant l'intégration CRYPTONEO, la signature s'effectuait soit par dépôt d'une image de signature manuscrite, soit par une simple action de validation dans l'application — approche jugée insuffisante en termes de preuve et de sécurité.

Les objectifs de l'intégration sont les suivants : garantir l'identité du signataire (certificat numérique nominatif adossé aux vérifications d'identité ONECI de la plateforme), sécuriser le consentement par un code OTP à usage unique transmis par SMS ou e-mail, conserver une piste d'audit complète de chaque opération, et archiver automatiquement les documents signés au sein de l'infrastructure Supabase.

2.2 La plateforme CRYPTONEO / ANSUT

CRYPTONEO expose une API REST de signature électronique dont la racine est configurée par variable d'environnement (valeur de production : signature.ansut.ci ; le code embarque une valeur de démonstration de secours, ansut.cryptoneoplatforms.com/esignaturedemo). L'authentification s'effectue en deux temps : un appel au point d'entrée /user/auth avec les clés d'application (appKey, appSecret) retourne un jeton JWT porteur, valable environ trente minutes, à présenter ensuite en en-tête Authorization: Bearer sur chaque appel métier.

Particularité importante du contrat d'interface : CRYPTONEO renvoie la plupart des erreurs métier avec un code HTTP 200 et un code métier dans le corps de la réponse. La logique applicative doit donc toujours inspecter ce code (champ statusCode ou code) et non le seul statut HTTP. Les codes de succès identifiés sont 200 (opération générique), 7000 (génération de certificat, liste d'utilisateurs), 7002, 7003 (signature par lots) et 7004 (vérification). Le code 0 ainsi que les codes supérieurs ou égaux à 8000 (par exemple 8006, paramètres invalides) signalent des échecs.

2.3 Périmètre de l'intégration

Sont couverts par l'intégration : la génération et le cycle de vie des certificats numériques des utilisateurs ; la signature électronique des baux avec double signature propriétaire / locataire ; la signature des mandats de gestion avec double signature propriétaire / agence ; la synchronisation des alias de certificats côté back-office ; et les rappels automatiques des signatures en attente. Ne relèvent pas de ce périmètre, bien qu'ils partagent des infrastructures communes : les paiements (Intouch), la vérification d'identité KYC (RNPP / ONECI, utilisée en amont pour enrichir le certificat) et la messagerie (ANSUT messaging).

3. Architecture technique

3.1 Principe directeur : secret côté serveur uniquement

Les clés d'application CRYPTONEO ne sont lues que dans des variables d'environnement serveur (process.env côté Next.js, Deno.env côté Edge Functions). Le client navigateur — comme l'application mobile Capacitor — ne dialogue jamais directement avec CRYPTONEO : il appelle soit une route API Next.js, soit une Edge Function Supabase, en

présentant son propre jeton d'authentification Supabase (JWT utilisateur). C'est la couche serveur qui porte l'identité CRYPTONEO, le cache du jeton JWT et la gestion des erreurs.

Deux runtimes serveur coexistent et se partagent le travail : les routes API Next.js gèrent les flux riches (baux, mandats, OTP locaux, synchronisation d'alias) avec accès direct à la base via le client service_role ; les Edge Functions Supabase exposent les primitives de signature au navigateur en masquant les clés (génération de certificat, envoi d'OTP, signature, vérification). Cette séparation est résumée dans le tableau 1.

Tableau 1 — Chaîne de composants de l'intégration CRYPTONEO

Couche	Technologie	Rôle
Interface utilisateur	React / Next.js (web) · Capacitor (mobile)	Wizards de signature, collecte du consentement et de la photo, saisie de l'OTP, suivi du statut.
API Next.js	Route Handlers (src/app/api)	Flux métier baux et mandats, OTP locaux, synchronisation des alias, accès base via client service_role.
Edge Functions Supabase	Deno (supabase/functions)	Primitives CRYPTONEO sécurisées : certificat, OTP, signature, vérification. Masquent appKey / appSecret.
Contrat partagé	cryptoneo-contract.ts (pur TypeScript)	Source de vérité commune : codes de succès, types d'API, normalisation du genre (DAT-004).
CRYPTONEO / ANSUT	API REST externe	Authentification par clés, génération de certificats, OTP, signature par lots, vérification, récupération.

3.2 Le contrat partagé (initiative DAT-004)

Historiquement, la logique CRYPTONEO était dupliquée entre la bibliothèque Node (src/lib/cryptoneo.ts) et la bibliothèque Deno (supabase/functions/_shared/cryptoneo.ts), et les deux copies avaient divergé sur les codes de succès et le traitement du code 0. L'initiative DAT-004 a extrait toute la logique pure — c'est-à-dire sans dépendance d'exécution — dans un contrat unique, supabase/functions/_shared/cryptoneo-contract.ts, importé et réexporté par les deux runtimes.

Ce contrat contient : la constante CRYPTONEO_TOKEN_TTL_MS (trente minutes), l'ensemble CRYPTONEO_SUCCESS_CODES, les fonctions isCryptoneoSuccess et

normalizeCryptoneoGender, ainsi que les neuf formes de réponse de l'API (CryptoneoAuthResponse, CryptoneoCertificatResponse, CryptoneoOtpResponse, CryptoneoSignResponse, CryptoneoVerifyResponse, CryptoneoUser, CryptoneoUsersResponse, CryptoneoSignedFileResponse, SignRequestItem). La règle d'évolution est stricte : toute modification du contrat se fait d'abord dans ce fichier ; un diff qui ajoute un code de succès dans un seul runtime est considéré comme un bug. Le document docs/DEDUP-LIB-EDGE-FUNCTIONS.md fait foi pour cette gouvernance.

3.3 Gestion du jeton JWT

Chaque runtime conserve le jeton CRYPTONEO en mémoire avec une durée de vie de trente minutes (CRYPTONEO_TOKEN_TTL_MS) et une marge de sécurité de soixante secondes avant renouvellement. En cas de réponse 401 en cours d'usage, le cache est invalidé puis l'appel est rejoué une seule fois après nouvelle authentification (mécanisme cryptoneoFetch avec retry unique).

L'Edge Function cryptoneo-auth joue le rôle de courtier de jeton pour les autres fonctions : elle expose le jeton aux fonctions internes authentifiées, applique un cache de cinquante-cinq minutes et une politique de trois tentatives avec back-off exponentiel en cas d'échec d'authentification auprès de CRYPTONEO. Les identifiants invalides (401 / 403 de CRYPTONEO) sont remontés immédiatement comme une erreur de configuration.

```
POST {CRYPTONEO_API_URL}/user/auth
Corps : { "appKey": "…", "appSecret": "…" }
Réponse : { "data": { "token": "<JWT>" }, "statusCode": 200 }
Cache mémoire : 30 min (TTL contrat) — marge 60 s — retry unique sur 401
```

4. Configuration et prérequis

4.1 Variables d'environnement

Les variables de signature sont documentées dans .env.local.example (côté Next.js) et dans docs/ENVIRONNEMENT.md (référentiel global). Les deux runtimes lisent les mêmes noms de secrets : ils doivent donc être maintenus strictement synchrones entre le fichier .env.local de l'application et les secrets des Edge Functions Supabase (Dashboard → Edge Functions → Secrets, ou CLI supabase secrets set).

Tableau 2 — Variables d'environnement de l'intégration

Variable	Runtime	Rôle et valeur de référence
CRYPTONEO_API_URL	Next.js	Racine de l'API de signature. Production : https://signature.ansut.ci. Le code embarque une URL de démonstration par défaut.
CRYPTONEO_BASE_URL	Edge Functions (Deno)	Même racine, nommée différemment côté Deno dans les fonctions historiques. Obligatoire — son absence arrête la fonction.
CRYPTONEO_APP_KEY	Next.js + Edge Functions	Clé d'application fournie par CRYPTONEO / ANSUT. Secret 🔴 — jamais côté client.
CRYPTONEO_APP_SECRET	Next.js + Edge Functions	Secret d'application associé. Secret 🔴 — rotation documentée dans docs/RUNBOOK-ROTATION-SECRETS.md.
CRON_SECRET	Edge Functions (Deno)	Secret Bearer des fonctions planifiées (dont les rappels de signature en attente). Obligatoire en production.

4.2 Modèle de données

L'intégration s'appuie sur des tables dédiées et sur des colonnes ajoutées aux tables métier. Les migrations correspondantes sont livrées dans supabase/migrations/. Les tables signature_aliases et otp_codes activent la sécurité RLS avec des politiques restreignant chaque utilisateur à ses propres lignes (auth.uid()).

Tableau 3 — Tables et colonnes utilisées par la signature électronique

Table / colonnes	Rôle
signature_aliases	Alias de certificat CRYPTONEO par utilisateur (alias_certificat unique, identité, type de pièce, hash de pièce, données de certificat). Un utilisateur = un alias (user_id unique). RLS : lecture / écriture limitées au propriétaire de la ligne.

Table / colonnes	Rôle
otp_codes	Codes à usage unique de tous les flux (connexion, vérification e-mail, réinitialisation, signature de bail). Colonnes clés : code, type (enum otp_type, dont BAIL_SIGNATURE), expires_at, is_used. RLS : accès limité au détenteur.
digital_certificates	Certificats numériques générés : certificate_id (alias CRYPTONEO), certificate_data (réponse complète de l'API), certificate_status, expires_at (un an).
lease_contracts.cryptoneo_operation_id, cryptoneo_signature_status, cryptoneo_initiated_at	Suivi de l'opération de signature d'un bail : identifiant d'opération CRYPTONEO, statut (processing…), horodatage d'initiation.
mandats.cryptoneo_operation_id, contract_url, owner_signature_image, agency_signature_image	Suivi de signature du mandat de gestion (migration 20260910000001, ajoutée pour corriger une erreur « column does not exist »).
agency_mandates.cryptoneo_operation_id, cryptoneo_signature_status, owner_signed_at, agency_signed_at	Suivi de la double signature propriétaire / agence des mandats d'agence.

4.3 Checklist de mise en configuration

1. Déclarer CRYPTONEO_API_URL, CRYPTONEO_APP_KEY et CRYPTONEO_APP_SECRET dans .env.local (jamais versionné) et dans les secrets des Edge Functions.
2. Déclarer CRYPTONEO_BASE_URL côté Edge Functions (nom utilisé par les fonctions historiques) et vérifier la cohérence des racines d'API entre les deux runtimes.
3. Vérifier dans supabase/config.toml que les fonctions métier de signature sont déployées avec verify_jwt = true (voir section 8.1).
4. Appliquer les migrations du modèle de données (signature_aliases, otp_codes, colonnes cryptoneo_* des tables métier).
5. Exécuter les tests de parité du contrat (bunx tsc --noEmit, cas isCryptoneoSuccess / normalizeCryptoneoGender — section 9.3).
6. Tester l'authentification en appelant la fonction cryptoneo-auth, puis un parcours de signature complet sur l'environnement de démonstration.

5. Cycle de vie de la signature électronique

Le flux complet comporte cinq étapes, illustrées par la figure 1 : la génération du certificat numérique (une fois par utilisateur), l'envoi du code OTP, la signature par lots du document, la vérification du statut de l'opération et l'archivage du document signé. Les sections suivantes détaillent chaque étape, puis les deux déclinaisons métier (baux et mandats).



Figure 1 — Cycle de vie de la signature électronique CRYPTONEO

5.1 Étape 1 — Génération du certificat numérique

Le certificat numérique est l'identité de signature de l'utilisateur auprès de CRYPTONEO. La fenêtre modale ElectronicSignatureModal (pilotée par le hook useElectronicSignature) collecte le genre, la photo de la pièce d'identité et le consentement explicite avec sa date. Le service client cryptoNeoService.generateCertificate calcule d'abord le hachage SHA-256 de la photo sur les données binaires (conformément à la documentation CRYPTONEO), convertit la photo en base64, puis appelle l'Edge Function cryptoneo-generate-certificate avec un délai d'attente de soixante secondes.

Côté serveur, la fonction authentifie l'appelant (JWT Supabase), récupère le profil (full_name, phone) et, le cas échéant, les données de vérification ONECI (user_verifications). Le hashPiece transmis à CRYPTONEO suit une cascade documentée : hachage de la photo fourni par le client (cas privilégié), sinon hachage recalculé côté serveur depuis le base64, sinon hachage du numéro de CNI, en dernier recours hachage de l'adresse e-mail — chaque repli étant journalisé comme dégradation. La requête vers /generateCert/generateCertificat transmet les champs attendus par la plateforme :

```
POST {CRYPTONEO_BASE_URL}/generateCert/generateCertificat
{ "firstName": "…", "lastName": "…", "email": "…", "phone": "…",
  "organisation": "CRYPTONEO", "typePiece": "CNI", "hashPiece": "<sha256>",
  "gender": "Homme|Femme", "consent": true, "consentDate": "…", "base64":
"…" }
Succès attendu : statusCode 7000 → data.certificatId + data.alias
```

En cas de succès, la fonction enregistre le certificat dans digital_certificates (statut active, expiration à un an), notifie l'utilisateur (type certificate_generated) et trace l'opération dans admin_audit_logs. Le nom de l'organisation est fixé à la valeur littérale CRYPTONEO, exigée par la plateforme. Le genre est normalisé par le contrat partagé

(normalizeCryptoneoGender) qui accepte les variantes homme / m / male / masculin et femme / f / female / feminin, avec un défaut à Homme.

Parallèlement, l'API de liste /generateCert/users alimente la route d'administration /api/signature/sync-aliases, qui resynchronise la table signature_aliases depuis la plateforme — cette route est le seul consommateur applicatif de la fonction isCryptoneoSuccess côté Next.js, alignée volontairement sur le comportement des Edge Functions.

5.2 Étape 2 — Demande et envoi du code OTP

Deux mécanismes d'OTP coexistent selon le flux. Le premier s'appuie directement sur CRYPTONEO : l'Edge Function cryptoneo-send-otp transmet à /otp/send un canal (SMS ou MAIL) et un destinataire — le numéro de téléphone doit respecter le format international E.164 (motif ^\+[1-9]\d{7,14}$). Le second, propre aux baux, est un OTP local : la route POST /api/leases/[id]/request-sign-otp génère un code à six chiffres, valable vingt-quatre heures, stocké dans otp_codes avec le type BAIL_SIGNATURE, après vérifications d'usage (propriétaire ou locataire du bail, bail au statut PENDING_SIGNATURE, utilisateur n'ayant pas déjà signé). Si un code encore valide existe, il est renvoyé tel quel afin d'éviter les doublons ; le code est également retourné dans la réponse pour affichage dans l'interface.

5.3 Étape 3 — Signature du document (signFileBatch)

L'Edge Function cryptoneo-sign-document orchestre la signature. Elle enchaîne sept contrôles et actions dans un ordre strict : validation de l'OTP fourni (recherche dans otp_codes d'un code non expiré associé à l'appelant — le marqueur isOtpError qualifie les échecs), récupération du certificat actif de l'utilisateur dans digital_certificates (les certificats localement générés, sans identifiant CRYPTONEO, sont refusés avec le marqueur isLocalCertificate et exigent une régénération), obtention du jeton JWT via cryptoneo-auth, construction de la requête de signature, appel de /sign/signFileBatch, puis traitements post-signature.

La requête de signature porte l'alias du certificat (aliasCertificat) et un tableau signRequest décrivant chaque document. La plateforme attend la structure SignRequestItem du contrat partagé ; les valeurs par défaut projet sont positionnées côté serveur : positionImage 150,200, pageImage 1, lieuSignature Abidjan, motifSignature Signature électronique. Le champ otp est transmis vide : l'OTP a déjà été validé localement, et le marqueur d'utilisation (used = true) n'est posé qu'après succès de la signature, ce qui autorise les réessais en cas d'erreur intermédiaire.

Le succès est signifié par le code métier 7003 accompagné d'un operationId. La fonction marque alors l'OTP comme utilisé, enregistre l'identifiant d'opération sur le contrat concerné (cryptoneo_operation_id, statut processing, horodatage cryptoneo_initiated_at), notifie les

deux parties (type contract_signature_processing) et écrit une entrée d'audit (contract_signature_initiated). Tout autre code métier est traité comme un échec et remonté avec son message d'origine.

5.4 Étape 4 et 5 — Vérification et archivage

La signature étant asynchrone, l'Edge Function cryptoneo-verify-signature interroge /sign/verifySignedBatch avec l'operationId pour connaître le statut réel de l'opération — le succès est signifié par le code métier 7004 et un résultat par document. Pour chaque document signé avec succès (code 7000 dans les résultats), la fonction télécharge le PDF signé depuis l'URL de téléchargement fournie par la plateforme, le verse dans le bucket privé contract-documents de Supabase Storage sous la forme contracts/{id}_signed_{horodatage}.pdf, puis met à jour le contrat avec l'URL du document signé. Le point d'entrée /sign/getSignedFile complète le dispositif pour la récupération ciblée d'un fichier signé, exploitée notamment par le composant certificate-retrieval.

5.5 Déclinaison baux : signature à deux avec activation du bail

Le bail requiert la signature du propriétaire puis du locataire (ou inversement). L'interface locataire (lease-detail.tsx) implémente un dialogue en deux temps : obtention du code OTP (bouton Obtenir un code OTP) puis confirmation (Confirmer la signature), le bouton de signature restant inactif tant que le code à six chiffres n'est pas saisi. La route POST /api/leases/[id]/sign authentifie l'appelant, vérifie son rôle sur le bail (propriétaire ou locataire), contrôle le statut et l'OTP, puis enregistre la signature. Lorsque les deux parties ont signé, le bail passe au statut ACTIVE et la fonction utilitaire notifyLeaseActivated (src/lib/notify.ts) notifie simultanément les deux parties que le bail est actif et signé électroniquement. Le même double déclenchement de notification est appliqué sur l'action de signature historique (PATCH /api/leases/[id]).

5.6 Déclinaison mandats : double signature propriétaire / agence

Le mandat de gestion suit le même schéma avec un couple de signataires propriétaire / agence. La route POST /api/mandats/[id]/request-sign-otp réserve la demande d'OTP au propriétaire, pour un mandat au statut DRAFT ou PENDING_SIGNATURE n'ayant pas déjà été signé par ses soins ; le canal est déterminé selon les coordonnées disponibles (e-mail par défaut, SMS si numéro E.164 valide). La route POST /api/mandats/[id]/sign enregistre la signature du propriétaire ou de l'agence, écrit l'image de signature correspondante (owner_signature_image ou agency_signature_image) et l'identifiant d'opération CRYPTONEO, puis génère le PDF du mandat signé (route /api/mandats/[id]/pdf). Les colonnes utilisées ont été ajoutées par la migration 20260910000001, qui corrigeait une erreur systématique « column does not exist » lors des tentatives de signature.

Le service src/services/mandates/cryptoneoSignatureService.ts, hérité de l'époque Vite, implémente une variante de ce flux (expiration de l'opération à soixante-douze heures, OTP InTouch). Il est documenté ici pour complétude mais doit être considéré comme du code legacy : l'axe actif est le couple de routes API Next.js décrit ci-dessus, et l'alignement du service historique (variables import.meta.env) figure parmi les recommandations d'évolution (section 10.3).

6. Cartographie des composants du projet

6.1 Edge Functions Supabase

Tableau 4 — Edge Functions de l'intégration (supabase/functions/)

Fonction	Rôle	Authentification
cryptoneo-auth	Courtier de jeton JWT CRYPTONEO (cache 55 min, 3 tentatives avec back-off).	JWT Supabase requis ; consommée aussi en interne par les autres fonctions.
cryptoneo-generate-certificate	Génération du certificat numérique (profil, ONECI, photo, consentement) et enregistrement.	verify_jwt = true
cryptoneo-send-otp	Envoi d'un OTP CRYPTONEO par SMS (E.164) ou e-mail.	verify_jwt = true
cryptoneo-sign-document	Validation OTP locale + signature par lots /sign/signFileBatch + suivi contrat.	verify_jwt = true
cryptoneo-verify-signature	Statut de l'opération (/sign/verifySignedBatch), téléchargement et stockage du PDF signé.	verify_jwt = true
sign / sign-verify / sign-send-otp / signed-file	Primitives historiques de signature, de vérification, d'OTP de signature et de récupération de fichier signé (consomment le contrat partagé).	verify_jwt = true
signature-auth	Variante d'authentification signature (ancienne).	Passage à verify_jwt = true planifié (commenté dans config.toml, voir runbook SEC-008).

Fonction	Rôle	Authentification
remind-pending-lease-signatures	Rappels quotidiens des baux en attente de signature (relance à 48 h, idempotent via reminder_sent_at).	Bearer CRON_SECRET (fonction planifiée).

6.2 Routes API Next.js

Tableau 5 — Routes API Next.js liées à la signature (src/app/api)

Route	Méthode	Rôle
/api/leases/[id]/request-sign-otp	POST	Génération / réémission de l'OTP local de signature du bail (type BAIL_SIGNATURE, 24 h).
/api/leases/[id]/sign	POST	Signature du bail par le propriétaire ou le locataire ; activation et double notification à l'activation.
/api/mandats/[id]/request-sign-otp	POST	Demande d'OTP réservée au propriétaire pour la signature du mandat.
/api/mandats/[id]/sign	POST	Signature du mandat (propriétaire / agence), images de signature, operationId, PDF signé.
/api/mandats/[id]/pdf	GET	Génération / récupération du PDF du mandat.
/api/signature/auth	POST	Passerelle d'authentification CRYPTONEO côté Next (consomme src/lib/cryptoneo.ts).
/api/signature/generate-certificate	POST	Génération de certificat via la bibliothèque Node (miroir de l'Edge Function).
/api/signature/send-otp · /sign · /verify · /signed-file	POST / GET	Primitives de signature côté Next : envoi OTP, signature, vérification, fichier signé.
/api/signature/sync-aliases	POST	Synchronisation admin de signature_aliases depuis /generateCert/users (isCryptoneoSuccess).

Route	Méthode	Rôle
/api/certificates/check	GET	Contrôle de l'existence / validité du certificat de l'utilisateur courant.

6.3 Bibliothèques, services et hooks

Tableau 6 — Bibliothèques et services de la signature

Fichier	Runtime	Rôle
supabase/functions/_shared/ cryptoneo-contract.ts	Aucun (pur)	Contrat partagé : codes de succès, types d'API, normalisation du genre. Source de vérité (DAT-004).
supabase/functions/_shared/ cryptoneo.ts	Deno	Couche runtime Deno : secrets Deno.env, cache du jeton, cryptoneoFetch / cryptoneoFetchJson.
src/lib/cryptoneo.ts	Node / Next	Couche runtime Node : process.env, cache du jeton, cryptoneoFetch ; réexporte le contrat. Serveur uniquement.
src/services/cryptonoe/ cryptonoe.service.ts	Client	CryptoNeoService (singleton) : appels aux Edge Functions avec JWT Supabase, utilitaires de hachage SHA-256 et de conversion base64.
src/services/mandates/ cryptoneoSignatureService.ts	Client (legacy Vite)	Variante historique du flux mandats (OTP InTouch, expiration 72 h) — à aligner sur les routes API (section 10.3).
src/services/contracts/ signatureService.ts	Client	Signature image (canvas) des contrats : enregistrement, contrôle de signature existante, notifications.
src/hooks/contract/ useElectronicSignature.ts	Client	Machine à états de la signature : idle, collect_data, generating_cert, waiting_otp, signing, completed, error.

Fichier	Runtime	Rôle
src/lib/notify.ts	Node / Next	notifyLeaseActivated : notification simultanée des deux parties à l'activation du bail.

6.4 Composants d'interface utilisateur

Côté interface, les composants principaux sont : ElectronicSignatureModal (src/shared/ui/electronic-signature/) qui orchestre l'assistant complet de signature — collecte des données, choix du genre, dépôt de la photo avec prévisualisation, consentement daté, canal d'OTP (SMS ou e-mail) et suivi des étapes du hook ; CryptoNeoSignature (src/shared/ui/) qui gère la saisie de l'OTP avec compte à rebours d'expiration et renvoi de code ; DigitalCertificateCard et certificate-retrieval qui présentent l'état du certificat et permettent de récupérer un document signé ; ElectronicSignatureGated et FeatureGate qui conditionnent l'accès à la signature selon les fonctionnalités activées ; ainsi que les écrans métier SignMandateWithOTPPage, SignMandatePage et les panneaux MandateCard / MandateDetailPanel côté agence.

7. Endpoints CRYPTONEO consommés

Le tableau 7 récapitule les points d'entrée de l'API CRYPTONEO utilisés par la plateforme, leur usage précis dans le projet et le code métier de succès attendu. Les formes de requête et de réponse sont figées dans le contrat partagé (types Cryptoneo*Response).

Tableau 7 — Endpoints CRYPTONEO / ANSUT consommés

Endpoint	Méthode	Usage dans le projet	Code succès
/user/auth	POST	Obtention du JWT (appKey + appSecret), cache trente minutes.	200
/generateCert/ generateCertificat	POST	Génération du certificat numérique de l'utilisateur (photo, CNI, consentement).	7000
/generateCert/users	GET	Liste des certificats / alias — synchronisation signature_aliases (admin).	7000
/otp/send	POST	Envoi du code OTP par SMS ou e-mail.	200

Endpoint	Méthode	Usage dans le projet	Code succès
/sign/signFileBatch	POST	Signature par lots des documents (aliasCertificat + signRequest[]).	7003
/sign/verifySignedBatch	POST	Vérification du statut de l'opération (operationId).	7004
/sign/getSignedFile	GET	Récupération d'un document signé (composant certificate-retrieval).	200

7.1 Format de la requête de signature (SignRequestItem)

Chaque élément de signRequest décrit un document à signer. Le format attendu par la plateforme et les valeurs par défaut posées par l'Edge Function cryptoneo-sign-document sont les suivants :

Tableau 8 — Champs de SignRequestItem (contrat partagé)

Champ	Défaut projet	Description
codeDoc	—	Identifiant unique du document dans l'opération (ex. identifiant du bail).
urlDoc / hashDoc	—	URL d'accès au PDF et hachage SHA-256 du document.
visibiliteImage / urlImage / hashImage	true	Image de signature visible : URL et hachage de l'image.
positionImage	150,200	Position de l'image de signature sur la page (x,y).
pageImage	1	Page porteuse de la signature.
messageImage	true	Affichage du message de signature.
lieuSignature	Abidjan	Lieu de signature inscrit sur le certificat.
motifSignature	Signature électronique	Motif de signature inscrit sur le certificat.

Le frontend transmet un format simplifié (SignRequestInputItem : fileName, base64, informations du signataire, position facultative) que la couche serveur convertit au format

SignRequestItem — le fichier à signer est hébergé et haché côté serveur, ce qui garantit la cohérence du hachage signé.

8. Sécurité

8.1 Authentification des appels

Le durcissement SEC-008 a classé les vingt-six Edge Functions du projet et activé verify_jwt = true pour les quatorze fonctions métier, dont toutes les fonctions CRYPTONEO : la plateforme Supabase rejette ainsi (401) toute requête sans jeton JWT signé par Supabase. Seules les fonctions de webhook restent ouvertes par conception, et le passage de signature-auth à verify_jwt = true est planifié (entrées commentées dans supabase/config.toml, checklist du runbook docs/RUNBOOK-EDGE-FUNCTIONS-SECURITE.md). Le motif de résolution d'utilisateur commun (resolveUserFromRequest) accepte soit un JWT utilisateur valide, soit le couple Bearer service_role + en-tête x-user-id réservé aux appels serveur à serveur.

8.2 Cycle de vie de l'OTP

L'OTP est le second facteur du consentement. Les règles appliquées sont les suivantes : code à six chiffres, expiration à vingt-quatre heures (baux) ; réutilisation du code encore valide plutôt que génération d'un doublon ; validation locale systématique avant tout appel CRYPTONEO (code non expiré rattaché à l'appelant) ; marqueur d'utilisation posé uniquement après succès de la signature afin de permettre les réessais en cas d'erreur intermédiaire ; et politiques RLS limitant la lecture des codes à leur détenteur. Les flux d'authentification hors signature appliquent en outre un hachage du code (src/lib/otp-hash.ts).

8.3 Codes de retour CRYPTONEO

Le traitement uniforme des réponses est assuré par isCryptoneoSuccess (contrat partagé) : le code métier est lu dans code ou statusCode, les codes inconnus sont traités comme des erreurs, et le code 0 n'est jamais un succès — c'est la correction apportée par DAT-004 (voir section 9.2).

Tableau 9 — Codes métier CRYPTONEO et traitement projet

Code	Signification	Traitement dans le projet
200	Succès générique (authentification, OTP, fichier signé).	Succès.

Code	Signification	Traitement dans le projet
7000	Succès — certificat généré ; liste d'utilisateurs ; document signé dans un lot.	Succès (génération certificat, résultats par document).
7002	Succès — opération de signature.	Succès.
7003	Succès — signature par lots acceptée (operationId retourné).	Succès ; suivi contrat + notifications + audit.
7004	Succès — vérification d'opération.	Succès ; téléchargement et archivage du PDF signé.
0	Échec de l'opération.	Erreur (jamais traité comme succès — divergence historique corrigée).
8006	Paramètres invalides.	Erreur ; message de la plateforme remonté à l'appelant.
≥ 8000	Autres erreurs métier.	Erreur ; code et statusMessage conservés pour diagnostic.

8.4 Journalisation d'audit et notifications

Chaque étape significative laisse une trace : table admin_audit_logs pour la génération de certificat (certificate_generated) et l'initiation de signature (contract_signature_initiated, avec operationId) ; table notifications pour l'utilisateur (certificat généré, signature en cours de traitement, activation du bail). Les Edge Functions journalisent en clair dans les logs Supabase pour le diagnostic ; l'application Next reste sobre dans ses messages. Attention : certains logs de fonctions incluent des données personnelles (profils, tailles de photo) — la purge de ces verbeux logs en production figure parmi les recommandations (section 10.3).

9. Gouvernance du code et tests de parité

9.1 Règle d'or DAT-004

Toute logique pure — constantes, fonctions sans entrées / sorties d'exécution, types et formes d'API — réside dans supabase/functions/_shared/cryptoneo-contract.ts, sans aucune dépendance Deno, Node ni npm. Les runtimes l'importent et n'ajoutent que leur couche d'exécution : variables d'environnement, cache du jeton, fetch authentifié. Cette règle élimine les divergences de comportement entre l'application et les Edge Functions, source du bug historique décrit ci-dessous.

9.2 Divergences historiques corrigées

Tableau 10 — Divergences corrigées par le contrat partagé

Élément	Avant (src/lib)	Avant (_shared)	Après (contrat)
Codes de succès	{200, 7000, 7002}	{200, 7000, 7002, 7003, 7004}	{200, 7000, 7002, 7003, 7004}
Code 0	Traité comme succès (falsy) — bug	Traité comme erreur	Erreur
Codes 7003 / 7004	Traités comme erreurs	Traités comme succès	Succès
TTL du jeton	30 min (codé localement)	30 min (codé localement)	CRYPTONEO_TOKEN_TTL_MS (30 min, marge 60 s)

9.3 Tests de parité à rejouer

Le document docs/DEDUP-LIB-EDGE-FUNCTIONS.md (section 3.2) fixe les tests de parité à rejouer à chaque évolution du contrat : typage des deux côtés (bunx tsc --noEmit côté Next, deno check côté Deno lorsque disponible), puis cas de vérité sur les deux implémentations avec résultats identiques attendus :

Tableau 11 — Cas de test de parité (isCryptoneoSuccess et normalizeCryptoneoGender)

Entrée	Résultat attendu
{ } (aucun code)	true (succès par défaut)
{ code: 200 } · { code: 7000 } · { code: 7002 } · { code: 7003 } · { code: 7004 }	true
{ code: 0 }	false
{ code: 8006 } · { statusCode: 8006 }	false
{ code: "9999" } (code non reconnu)	false
normalizeCryptoneoGender : homme / M / male / masculin	Homme
normalizeCryptoneoGender : femme / F / FEMALE / feminin	Femme
normalizeCryptoneoGender : undefined	Homme (défaut)

10. Exploitation, dépannage et recommandations

10.1 Erreurs courantes et résolutions

Tableau 12 — Dépannage de l'intégration CRYPTONEO

Symptôme	Cause probable	Résolution
« CRYPTONEO_BASE_URL environment variable not set »	Secret absent côté Edge Functions.	Déclarer CRYPTONEO_BASE_URL (et les clés) dans les secrets Supabase puis redéployer.
« Cryptoneo auth failed (401/403) » ou « Invalid CryptoNeo credentials »	appKey / appSecret invalides ou révoqués.	Vérifier les clés auprès de CRYPTONEO ; procédure de rotation (docs/RUNBOOK-ROTATION-SECRETS.md).
« Code OTP invalide ou expiré » (isOtpError)	OTP expiré, déjà consommé ou erroné.	Demander un nouveau code ; vérifier l'expiration (24 h) et le destinataire.
« Certificat local non valide… » (isLocalCertificate)	Certificat généré hors CRYPTONEO (données sans certificatId).	Faire régénérer le certificat de l'utilisateur.
« Timeout : la génération du certificat prend trop de temps (>60 s) »	Plateforme lente ou photo volumineuse.	Réessayer ; vérifier la taille de la photo et l'état de la plateforme.
« Erreur réseau : vérifiez votre connexion ou les paramètres CORS »	Échec fetch côté navigateur.	Contrôler la connectivité et les en-têtes CORS des Edge Functions.
« Certificat ID manquant dans la réponse CryptoNeo »	Réponse inattendue de la plateforme.	Inspecter les logs de l'Edge Function (réponse complète journalisée).
« column does not exist » à la signature d'un mandat	Migration 20260910000001 non appliquée.	Appliquer la migration (colonnes contract_url, images de signature, cryptoneo_operation_id).

10.2 Rappels automatiques des signatures en attente

La fonction planifiée remind-pending-lease-signatures s'exécute quotidiennement à 08 h 00 UTC (pg_cron, entrée [functions.remind-pending-lease-signatures] de supabase/config.toml). Elle relance les baux demeurés en attente de signature depuis quarante-huit heures, en s'appuyant sur le marqueur idempotent reminder_sent_at afin

d'éviter les doublons de relance. Son appel exige l'en-tête Authorization: Bearer <CRON_SECRET> lorsque le secret est défini — configuration obligatoire en production.

10.3 Recommandations d'évolution

- Aligner le service mandats historique (cryptoneoSignatureService.ts, variables import.meta.env héritées de Vite) sur les routes API Next.js actives, puis le retirer ou le réécrire — il ne doit plus constituer un second chemin de signature.
- Réduire la verbosité des logs des Edge Functions en production (données de profils et extraits de photo journalisés) : conserver les marqueurs d'étape, supprimer les contenus sensibles.
- Finaliser le passage de signature-auth à verify_jwt = true après authentification de /api/signature/auth côté Next (checklist runbook SEC-008, section 6).
- Documenter et déployer l'Edge Function de callback (callBackUrl par défaut pointant vers cryptoneo-callback) afin de recevoir les notifications de fin de signature plutôt que de s'appuyer uniquement sur la vérification tirée (polling).
- Programmer la rotation périodique des secrets CRYPTONEO conformément à docs/RUNBOOK-ROTATION-SECRETS.md (aucune fuite constatée au scan du dépôt : clés jamais en clair dans le code ni dans l'historique).
- Étendre les tests de parité du contrat (section 9.3) à une exécution automatisée dans la CI (.github/workflows/ci.yml) à chaque évolution du fichier cryptoneo-contract.ts.

10.4 Documents de référence internes

- docs/DEDUP-LIB-EDGE-FUNCTIONS.md — contrat commun CRYPTONEO, divergences corrigées et tests de parité (DAT-004).
- docs/ENVIRONNEMENT.md — inventaire des variables d'environnement, section « Signature — CRYPTONEO / ANSUT ».
- docs/RUNBOOK-EDGE-FUNCTIONS-SECURITE.md — classification verify_jwt des 26 Edge Functions (SEC-008).
- docs/RUNBOOK-ROTATION-SECRETS.md — procédure de rotation des secrets fournisseurs.
- docs/RUNBOOK-CRONS.md — planification des fonctions, dont les rappels de signature.
- agent-ctx/1-cryptoneo-signature-agent.md et agent-ctx/2-otp-lease-signing.md — journaux d'implémentation des tâches d'origine.

Fin du document.